

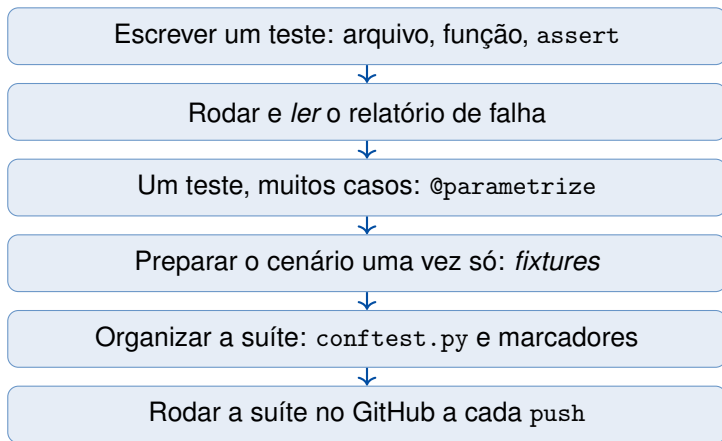
Testes Automatizados com pytest

Do `assert` solto à suíte que protege o seu código

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP — Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`



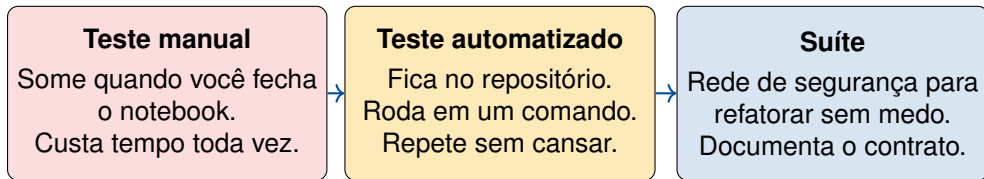
Por que automatizar testes

Problema:

Você roda a célula, olha o resultado, acha que está certo e segue em frente. Na próxima mudança, refaz tudo — ou pior, *não* refaz.

Solução:

Escrever a verificação **como código**. Ela fica no repositório, roda em um segundo e avisa quando algo que funcionava parou de funcionar.



Definição

Um **teste automatizado** é um trecho de código que executa outro trecho de código com entradas conhecidas e *verifica* se a saída é a esperada — falhando de forma ruidosa quando não é.

Intuição

O teste é uma *pergunta com resposta já conhecida*. Se o programa responde diferente, ou o programa mudou ou a sua expectativa estava errada. Nos dois casos você quer saber agora, não em produção.

O que é

pytest é um framework que **encontra** os seus testes, **executa** todos eles e **relata** quais passaram e quais falharam — com um diagnóstico legível de cada falha.

Por que ele e não `unittest`

Teste é função comum, verificação é o `assert` do Python. Sem classes obrigatórias, sem `assertEqual`, sem herança. Menos cerimônia significa mais testes escritos de fato.

Anatomia de um teste


```
1 def media(notas):  
2     """Media aritmetica de uma lista nao vazia de notas."""  
3     return sum(notas) / len(notas)  
4  
5 def aprovado(nota, minimo=6.0):  
6     """True se a nota alcanca a media minima de aprovacao."""  
7     return nota >= minimo
```

Listing 1: notas.py

Duas funções simples, sem estado, sem entrada do usuário — o cenário mais fácil possível para começar a testar.

```
1 from notas import media, aprovado
2
3 def test_media_de_tres_notas():
4     assert media([6.0, 7.0, 8.0]) == 7.0
5
6 def test_aprovado_com_nota_na_media():
7     assert aprovado(6.0) is True
8
9 def test_reprovado_abaixo_da_media():
10    assert aprovado(5.9) is False
```

Listing 2: test_notas.py

Três coisas apenas: importar, chamar, assert.



Nenhum registro, nenhuma lista de testes para manter: o nome é a inscrição.

```
$ pytest
collected 3 items
test_notas.py ... [100%]
===== 3 passed in 0.01s =====
```

Comando	Efeito
pytest	roda tudo que encontrar
pytest -v	mostra o nome de cada teste
pytest -q	saída curta
pytest -k media	só testes cujo nome casa com media
pytest test_notas.py::test_media	um teste específico
pytest -x	para na primeira falha

```
===== FAILURES =====
----- test_media_de_tres_notas -----

    def test_media_de_tres_notas():
>         assert media([6.0, 7.0, 8.0]) == 7.0
E         assert 21.0 == 7.0
E         + where 21.0 = media([6.0, 7.0, 8.0])

test_notas.py:4: AssertionError
===== 1 failed in 0.01s =====
```

A linha E mostra os *valores reais*: a função devolveu 21.0. Esqueceram de dividir pelo tamanho.

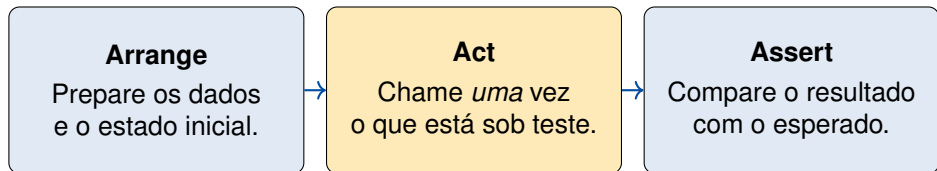
Introspecção de asserção

O `pytest` reescreve o `assert` antes de executá-lo para guardar os valores intermediários. Quando falha, ele reconstrói a expressão mostrando o que cada parte valia.

Consequência prática

Você não precisa de `assertEqual`, `assertTrue`, `assertAlmostEqual`. Escreva a condição em Python normal — o diagnóstico vem de graça.

A forma de todo teste: Arrange–Act–Assert



Se o seu teste tem dois *Act*, provavelmente são dois testes.

```
1 def test_media_ignora_ordem_das_notas():
2     notas_ordenadas = [5.0, 7.0, 9.0] # Arrange
3     notas_embaralhadas = [9.0, 5.0, 7.0]
4
5     resultado_a = media(notas_ordenadas) # Act
6     resultado_b = media(notas_embaralhadas)
7
8     assert resultado_a == resultado_b # Assert
```

As três fases separadas por linhas em branco: dá para ler o teste sem entender a implementação.

Nomes fracos

- test_1
- test_media
- test_funciona

Quando falham, você ainda precisa abrir o código.

Nomes que dizem o comportamento

- test_media_de_lista_vazia_falha
- test_aprovado_na_nota_exata
- test_conceito_A_a_partir_de_9

O relatório de falha vira uma frase.

```
1 import pytest
2 from notas import media
3
4 def test_media_de_lista_vazia_levanta_erro():
5     with pytest.raises(ZeroDivisionError):
6         media([])
```

O que este teste diz

Lançar exceção não é acidente: é *parte do contrato*. O teste registra que lista vazia deve falhar — e falharia se alguém resolvesse devolver 0 silenciosamente.

```
1 def test_media_com_dizima():
2     assert media([1.0, 2.0]) == 1.5           # exato: ok
3
4 def test_media_de_tres_notas_quebradas():
5     # 0.1 + 0.2 + 0.3 nao da exatamente 0.6 em binario
6     assert media([0.1, 0.2, 0.3]) == pytest.approx(0.2)
```

`pytest.approx` compara com tolerância relativa. Use sempre que o resultado passar por divisão ou soma de floats.

O que já conseguimos fazer

- Escrever `test_*.py` com funções `test_*` e `assert`
- Rodar a suíte e selecionar testes com `-v`, `-k`, `:`
- Ler o relatório de falha e descobrir o valor real
- Separar Arrange, Act e Assert
- Testar exceções com `pytest.raises` e floats com `pytest.approx`

Parametrização

```
1 def test_conceito_A(): assert conceito(9.5) == "A"  
2 def test_conceito_B(): assert conceito(8.0) == "B"  
3 def test_conceito_C(): assert conceito(6.5) == "C"  
4 def test_conceito_D(): assert conceito(4.0) == "D"
```

Quatro testes idênticos em estrutura. Se a função mudar de nome, são quatro edições. E o quinto caso ninguém lembra de acrescentar.

A solução: `@pytest.mark.parametrize`

```
1 @pytest.mark.parametrize("nota, esperado", [  
2     (9.5, "A"),  
3     (8.0, "B"),  
4     (6.5, "C"),  
5     (4.0, "D"),  
6 ])  
7 def test_conceito_por_faixa(nota, esperado):  
8     assert conceito(nota) == esperado
```

Uma função, quatro execuções independentes. Acrescentar um caso é acrescentar uma linha na tabela.

```
$ pytest -v
test_notas.py::test_conceito_por_faixa[9.5-A] PASSED
test_notas.py::test_conceito_por_faixa[8.0-B] PASSED
test_notas.py::test_conceito_por_faixa[6.5-C] FAILED
test_notas.py::test_conceito_por_faixa[4.0-D] PASSED
```

Por que isso importa

O caso que falhou não interrompe os outros, e aparece identificado no relatório. Com um `for` dentro de um único teste, você perderia essa informação na primeira falha.


```
1 @pytest.mark.parametrize("nota, esperado", [  
2     (10.0, True),  
3     (6.0, True),  
4     (5.99, False),  
5 ], ids=["nota-maxima", "exatamente-na-media", "logo-abaixo"])  
6 def test_aprovacao_nas_fronteras(nota, esperado):  
7     assert aprovado(nota) is esperado
```

Escolha os casos nas **fronteiras**: o valor exato do limite, logo abaixo, logo acima. É lá que os bugs moram.

O que aprendemos

- Uma tabela de casos substitui testes copiados
- Cada linha vira um teste independente no relatório
- `ids` dão nome legível a cada caso
- Priorize valores de fronteira e casos degenerados

Fixtures

O problema: o mesmo Arrange em todo teste

```
1 def test_media_da_turma():
2     turma = Turma("COMPO496")
3     turma.matricular("Ana", [8.0, 9.0])
4     turma.matricular("Bruno", [6.0, 5.0])
5     assert turma.media_geral() == 7.0
6
7 def test_aprovados_da_turma():
8     turma = Turma("COMPO496")           # de novo
9     turma.matricular("Ana", [8.0, 9.0])
10    turma.matricular("Bruno", [6.0, 5.0])
11    assert turma.aprovados() == ["Ana"]
```

```
1 @pytest.fixture
2 def turma_com_dois_alunos():
3     turma = Turma("COMP0496")
4     turma.matricular("Ana", [8.0, 9.0])
5     turma.matricular("Bruno", [6.0, 5.0])
6     return turma
7
8 def test_media_da_turma(turma_com_dois_alunos):
9     assert turma_com_dois_alunos.media_geral() == 7.0
10
11 def test_aprovados_da_turma(turma_com_dois_alunos):
12     assert turma_com_dois_alunos.aprovados() == ["Ana"]
```

O parâmetro do teste tem o *nome* da fixture — é assim que o pytest sabe o que injetar.

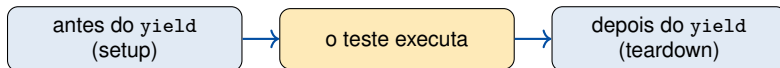
Definição

Uma **fixture** é uma função decorada com `@pytest.fixture` que produz um recurso pronto para o teste. O `pytest` a executa sob demanda e entrega o valor ao teste que a pede pelo nome.

Intuição

É o *Arrange* extraído para um lugar só, com nome. Assim como extrair uma função elimina código repetido, extrair uma fixture elimina preparação repetida.

```
1 @pytest.fixture
2 def arquivo_de_notas(tmp_path):
3     caminho = tmp_path / "notas.csv"
4     caminho.write_text("Ana,8.0\nBruno,5.0\n")
5     yield caminho          # o teste roda aqui
6     caminho.unlink()       # limpeza, mesmo se o teste falhar
```



Escopo	Executa	Use quando
function (padrão)	uma vez por teste	o teste modifica o recurso
class	uma vez por classe	testes agrupados compartilham
module	uma vez por arquivo	preparo caro e só de leitura
session	uma vez por execução	conexão, servidor, dado fixo

Cuidado

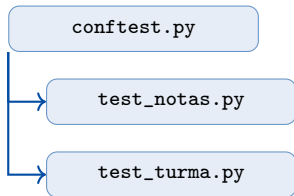
Escopo largo com recurso mutável cria dependência entre testes: um teste passa ou falha dependendo de quem rodou antes. Na dúvida, fique no padrão.

Fixture	Entrega
<code>tmp_path</code>	um diretório temporário exclusivo do teste
<code>capsys</code>	o que o código escreveu em <code>stdout/stderr</code>
<code>monkeypatch</code>	troca temporária de atributos, funções e variáveis

Basta pedir pelo nome no parâmetro do teste. Não precisa importar nem declarar nada.

```
1 def boletim(nome, media):
2     print(f"{nome}: {media:.1f}")
3
4 def test_boletim_formata_com_uma_casa(capsys):
5     boletim("Ana", 8.456)
6     saida = capsys.readouterr().out
7     assert saida == "Ana: 8.5\n"
```

Funções que só imprimem são difíceis de testar — por isso preferimos funções que *retornam*. Quando a impressão é o comportamento, `capsys` resolve.



Fixtures definidas em `conftest.py` ficam disponíveis para todos os testes daquele diretório e dos subdiretórios — **sem import**.

É o lugar certo para o cenário que mais de um arquivo de teste precisa.

Organizando a suíte

```
1 @pytest.mark.skip(reason="funcionalidade ainda nao implementada")
2 def test_exporta_boletim_em_pdf():
3     ...
4
5 @pytest.mark.skipif(sys.version_info < (3, 10),
6                     reason="usa match/case")
7 def test_classifica_com_match():
8     ...
9
10 @pytest.mark.xfail(reason="bug conhecido #42")
11 def test_media_de_lista_gigante():
12     ...
```

Sempre com reason: um teste pulado sem justificativa é um teste esquecido.

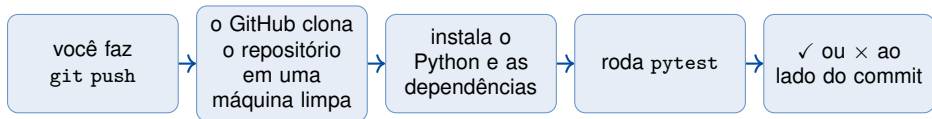
Características

- **Rápido** — se demora, ninguém roda
- **Independente** — passa sozinho e em qualquer ordem
- **Determinístico** — mesma entrada, mesmo resultado, sempre
- **Focado** — verifica um comportamento; o nome diz qual
- **Legível** — dá para entender sem abrir a implementação

Não teste

Detalhes internos que você quer poder mudar, nem a biblioteca padrão. Teste o *contrato*, não a implementação.

Integração contínua



A suíte só protege quem a executa — e a hora de esquecer é justamente a da pressa. No servidor ela roda sempre, em uma máquina onde existe apenas o que você declarou: a biblioteca instalada há seis meses e nunca anotada vira uma falha vermelha, não uma surpresa na véspera da entrega.


```
# .github/workflows/testes.yml
name: testes
on: [push]
jobs:
  testes:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install pytest
      - run: pytest -q
```

on diz *quando* rodar; runs-on, em que máquina; uses traz uma ação pronta; run é um comando de terminal. O caminho é a inscrição: todo .yaml em .github/workflows/ já é um *workflow*.

```
Run pytest -q
collected 12 items
test_notas.py ..... [100%]
===== 12 passed in 0.06s =====
```

- Um ✓ (ou ×) ao lado do commit — na lista de commits e dentro do *pull request*
- A aba **Actions** guarda o log completo de cada execução
- Quando quebra, o GitHub avisa por e-mail quem fez o push

O log traz o mesmo relatório do pytest que você já sabe ler: o teste que falhou, a linha e o valor real.

```
.github/workflows/*.yaml
```

1. onde o arquivo mora

```
on: [push]
```

2. quando a suíte roda

```
run: pytest
```

3. o que ela executa

Com isso, o verde deixa de ser algo que alguém lembra de conferir: passa a ser um fato registrado em cada commit, e o projeto pode exigí-lo antes de aceitar qualquer contribuição.

`test_*.py + def test_* + assert` — e o `pytest` acha sozinho

Leia o relatório de falha: ele mostra o valor real

`@parametrize` para a tabela de casos, com foco nas fronteiras

Fixtures para o Arrange repetido, `conftest.py` para compartilhar

Um *workflow* curto roda a suíte no GitHub a cada `push`

Suíte verde é permissão para refatorar

No tutorial (notebook)

- Prever o resultado de um teste antes de rodá-lo
- Diagnosticar uma falha lendo apenas o relatório
- Converter quatro testes repetidos em um `@parametrize`
- Extrair uma fixture e usá-la em dois testes
- Escrever a sua própria suíte para um módulo novo



Maurício Aniche.

Testes Automatizados de Software: Um Guia Prático.
Casa do Código, São Paulo, 2015.



Kent Beck.

Test-Driven Development: By Example.
Addison-Wesley, 2003.



Martin Fowler.

Refactoring: Improving the Design of Existing Code.
Addison-Wesley, 2 edition, 2019.



Gerard Meszaros.

xUnit Test Patterns: Refactoring Test Code.
Addison-Wesley, 2007.



Brian Okken.

Python Testing with pytest: Simple, Rapid, Effective, and Scalable.
The Pragmatic Bookshelf, 2 edition, 2022.



pytest development team.

pytest documentation.

<https://docs.pytest.org/>.

Acesso em 2026.